

ENGINEERING CASE STUDY

SupportLens

Customer Support Chatbot Observability Platform

Senior Full-Stack Engineer (Python Focus)
Core Edge Solutions

CONFIDENTIAL

Time Limit: 24 Hours from Receipt

Overview

You are building SupportLens: a lightweight observability platform for a customer support chatbot. The system has two parts — a simple support chatbot and a dashboard that monitors every conversation the chatbot handles.

When a user sends a message to the chatbot, the query and response are logged as a trace. A second LLM call then classifies each trace by subject category. The observability dashboard displays all traces and aggregate statistics so an operator can see what customers are asking about and how the chatbot is performing.

TIME LIMIT	24 hours from receipt (expect ~4–6 hours of active work)
DELIVERY	GitHub repository + Loom video + sample pull request
STACK	Python backend (any framework). Any LLM API. Frontend of your choice. Any database.
EVALUATION	We review your repo, PR, and Loom video

How It Works

The end-to-end flow is simple:

1	User sends a message to the support chatbot
2	Chatbot calls an LLM to generate a response
3	The query + response pair is saved as a trace on the backend
4	A second LLM call classifies the trace into one of five categories
5	The trace (with its classification) appears on the observability dashboard

Classification Categories

Every trace must be classified into exactly one of the following five categories using an LLM:

Category	Description
----------	-------------

Billing	Questions about invoices, charges, payment methods, pricing, or subscription fees
Refund	Requests to return a product, get money back, dispute a charge, or process a credit
Account Access	Issues logging in, resetting passwords, locked accounts, or MFA problems
Cancellation	Requests to cancel a subscription, downgrade a plan, or close an account
General Inquiry	Anything that doesn't fit the above: feature questions, product info, how-to questions, etc.

The classification must be done by an LLM (not keyword matching or regex). We will evaluate the quality of your classification prompt.

Data Model

Your application must store and expose the following. You may add fields but these are the minimum:

Trace

Field	Type	Description
id	string	Unique identifier
user_message	string	The customer's input to the chatbot
bot_response	string	The chatbot's generated reply
category	enum	Billing Refund Account Access Cancellation General Inquiry
timestamp	datetime	When the conversation occurred
response_time_ms	int	How long the chatbot took to respond

What to Build

Component 1: Support Chatbot

A minimal customer support chatbot. This does not need to be sophisticated — it exists to generate traces for the observability platform.

- A simple chat interface where a user types a message and receives a response
- The response is generated by an LLM (OpenAI, Anthropic, or any provider)
- **System prompt:** instruct the LLM to act as a customer support agent for a SaaS billing platform. Keep it simple.
- Each query/response pair is sent to the backend and saved as a trace
- The chatbot does not need conversation history, memory, or multi-turn support. Each message is independent.

Component 2: Backend API

Endpoints

- **POST /traces** — receives a trace (user message + bot response), classifies it via LLM, saves it, and returns the trace with its classification
- **GET /traces** — returns all traces, most recent first. Support optional filtering by category.
- **GET /analytics** — returns aggregate stats: total traces, breakdown by category (count and percentage), average response time

Classification Logic

When a trace is received via POST /traces:

1. Take the user_message and bot_response
2. Send them to an LLM with a prompt that classifies the conversation into one of the five categories
3. The LLM must return exactly one category from: Billing, Refund, Account Access, Cancellation, General Inquiry
4. Save the trace with the assigned category

Important: The classification prompt matters. We will review it during the walkthrough. A well-crafted prompt that handles edge cases (e.g., a message about both billing and cancellation) is expected.

Component 3: Observability Dashboard

Build a single-page dashboard with two sections:

Section 1: Aggregate View

Summary statistics displayed at the top of the page:

- **Total traces** processed
- **Breakdown by category:** count and percentage for each of the five categories (displayed as cards, a chart, or both)
- **Average response time** across all traces

Section 2: Trace List

A table showing all traces, most recent first:

- **Columns:** timestamp, user message (truncated), bot response (truncated), category (color-coded badge), response time
- **Clicking a row** expands or opens the full trace detail showing the complete user message and bot response
- **Filter by category** — a dropdown or button group that filters the table to a single category

Seed Data

Your application must come pre-loaded with:

- **At least 20 traces** already classified, with a realistic spread across all five categories
 - **The chatbot must work live** — in your Loom video, type a message into the chatbot and show the trace appearing on the dashboard with its classification
-

Running Locally

Your project must run locally with a single command. Include clear setup instructions in your README (e.g., docker-compose up or a short list of steps). We will clone your repo and run it ourselves.

Evaluation Criteria

Dimension	Weight	What We Look For
Working Product	30%	Does the full flow work end-to-end? Can we type a message, get a response, and see the classified trace on the dashboard? Does it run locally without issues?
LLM Integration	25%	Is the classification prompt well-crafted? Does it reliably assign the correct category? Is the chatbot system prompt sensible? Is the LLM integration architecturally clean?
UI/UX Quality	25%	Is the dashboard polished and intuitive? Is the chatbot usable? Are the aggregate stats and trace list clear and well-designed?
Code & Architecture	20%	Clean, readable code. Sensible API design. Thoughtful schema. Proper error handling.

Out of Scope

- User authentication
- Multi-turn conversation memory in the chatbot
- Real-time WebSocket updates (polling or manual refresh is fine)
- Mobile responsiveness (desktop-only is fine)
- Test suites

Submission

Reply to the email from which you received this document with:

1. **GitHub repository** (public or with access granted). Must include a README with setup instructions.
2. **Sample pull request** in the repo. Build the main project on the main branch, then create a feature branch with a meaningful addition (e.g., a new filter, a UI improvement, a new endpoint) and open a PR against main with a clear title, description, and clean diff.
3. **Loom video link** (3–5 min, in English) showing the app running locally and walking through how it works

Note: You may use any AI coding tools, libraries, or frameworks. We care about the outcome. However, you will be expected to explain every decision during the live walkthrough — especially your LLM prompts and architectural choices.

Loom Video

Record a 3–5 minute Loom video in English. Show the app running locally: send a message to the chatbot, show the trace appearing on the dashboard with its classification, and briefly explain your approach. That's it.

Good luck. Build something you'd be proud to demo.